

Fintrack - Product & Technical Specification

Version: 1.0

Date: June 2026

Author: Muhammad Perdiansyah

Status: Draft

1. Project Overview

Fintrack is a self-hosted personal finance tracker designed to help users track all their FIAT assets (bank accounts, e-wallets, cash, gold, stocks) with comprehensive reporting and analytics.

Vision

To provide a simple, privacy-focused, and comprehensive financial tracking solution that gives users full control over their financial data without relying on third-party services.

Target Users

- Individuals who want to track personal finances
 - Users who value data privacy and self-hosting
 - People managing multiple asset types (cash, bank, e-wallet, gold, stocks)
-

2. Goals & Success Metrics

Primary Goals

- Enable users to track all FIAT transactions across multiple accounts
- Provide clear visibility into net worth and spending patterns

- Ensure data privacy through self-hosting

Success Metrics (v1)

- Users can successfully add and manage multiple accounts
 - Users can track income, expenses, and transfers
 - Users can view net worth and spending breakdown
 - 95%+ uptime when self-hosted
-

3. Scope

Version 1 (FIAT Focus)

In Scope: - User authentication (JWT) - Account management (bank, e-wallet, cash, gold, stocks) - Transaction management (income, expense, transfer) - Basic reporting (net worth, spending by category) - Web interface

Out of Scope (Future Versions): - Cryptocurrency portfolio tracking (v2) - Mobile app (v2) - Multi-currency support (v2) - Recurring transactions (nice-to-have v1) - Budget tracking (v2) - Bank API integration (v3)

4. Functional Requirements

4.1 Authentication

- **FR-AUTH-001:** Users can register with email and password
- **FR-AUTH-002:** Users can login and receive JWT token
- **FR-AUTH-003:** JWT token expires after configurable time
- **FR-AUTH-004:** Passwords must be hashed using bcrypt

4.2 Account Management

- **FR-ACC-001:** Users can create multiple accounts
- **FR-ACC-002:** Each account has: name, type, balance, currency (IDR)
- **FR-ACC-003:** Account types: Bank, E-Wallet, Cash, Gold, Stock Broker
- **FR-ACC-004:** Users can view list of all accounts

- **FR-ACC-005:** Users can edit account details
- **FR-ACC-006:** Users can soft-delete accounts (set inactive)

4.3 Transaction Management

- **FR-TRX-001:** Users can add income transactions
- **FR-TRX-002:** Users can add expense transactions
- **FR-TRX-003:** Users can transfer between accounts
- **FR-TRX-004:** Each transaction has: amount, category, description, date
- **FR-TRX-005:** Transfer transactions update both source and destination balances
- **FR-TRX-006:** Users can view transaction history
- **FR-TRX-007:** Users can filter transactions by date, account, category

4.4 Categories

- **FR-CAT-001:** System provides default categories (Food, Transport, Salary, etc.)
- **FR-CAT-002:** Users can create custom categories
- **FR-CAT-003:** Categories are tagged as Income or Expense type

4.5 Reporting

- **FR-REP-001:** Users can view total net worth (sum of all active accounts)
- **FR-REP-002:** Users can view spending breakdown by category
- **FR-REP-003:** Users can filter reports by date range (monthly)

5. Non-Functional Requirements

5.1 Performance

- **NFR-PERF-001:** API response time < 500ms for 95% of requests
- **NFR-PERF-002:** Dashboard loads in < 2 seconds

5.2 Security

- **NFR-SEC-001:** All passwords must be hashed using bcrypt
- **NFR-SEC-002:** JWT tokens must be signed with secret key
- **NFR-SEC-003:** All API endpoints (except auth) require valid JWT
- **NFR-SEC-004:** Database credentials must be stored in environment variables

5.3 Reliability

- **NFR-REL-001:** System uptime > 95%
- **NFR-REL-002:** Database backups automated daily

5.4 Usability

- **NFR-USA-001:** Web interface responsive (desktop & mobile web)
 - **NFR-USA-002:** All timestamps in user's timezone (Jakarta/UTC+7)
-

6. Tech Stack & Architecture

6.1 Backend

- **Language:** Go 1.21+
- **Framework:** Gin
- **Architecture:** Clean Architecture (Domain → Usecase → Repository → Handler)
- **Database:** PostgreSQL 15+
- **Migration:** golang-migrate
- **Auth:** JWT (using golang-jwt)

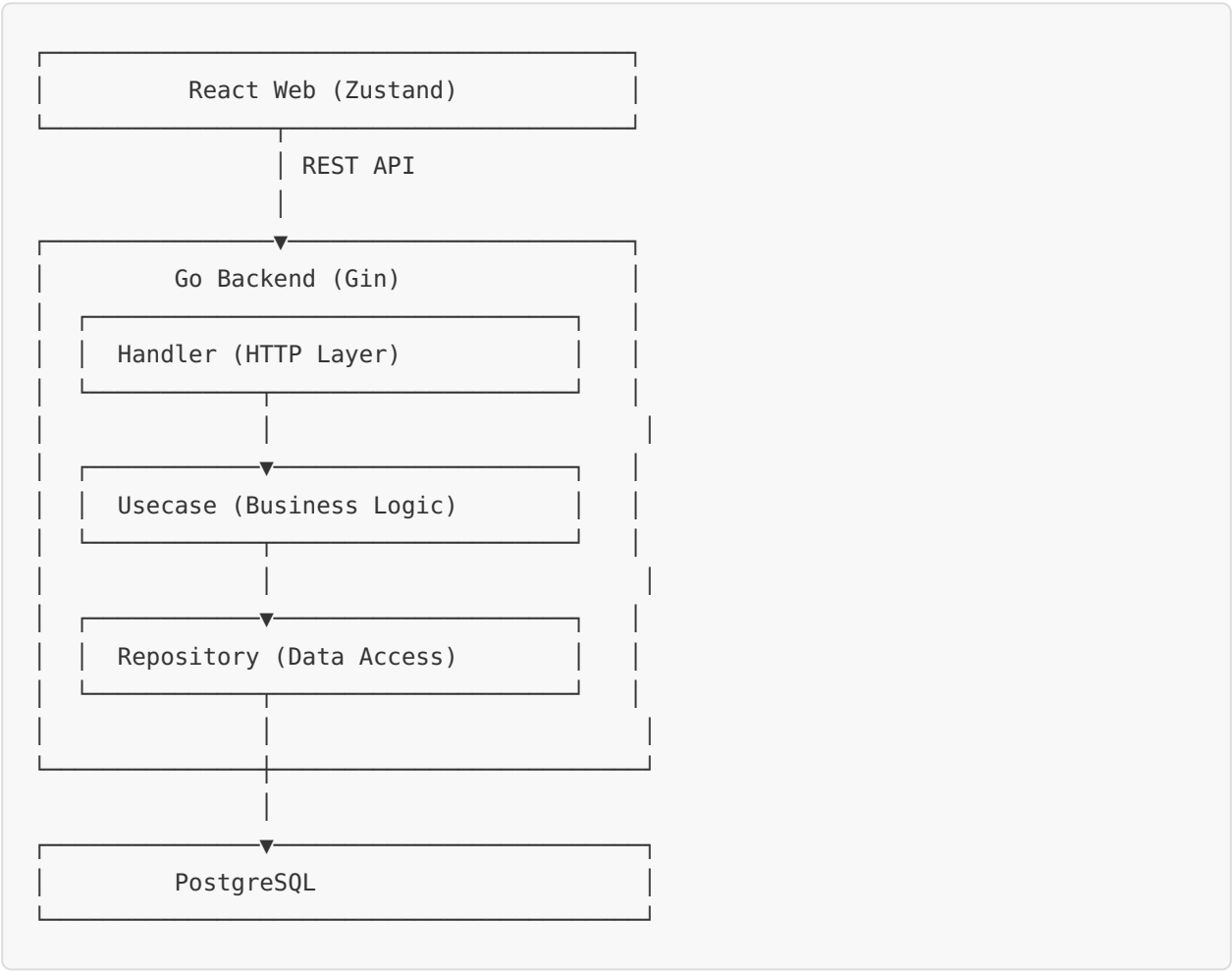
6.2 Frontend

- **Framework:** React 18+ with TypeScript
- **Styling:** Tailwind CSS
- **State Management:** Zustand
- **HTTP Client:** Axios
- **Routing:** React Router v6

6.3 Deployment

- **Containerization:** Docker + Docker Compose
- **Target:** Self-hosted on Proxmox VE server
- **Reverse Proxy:** Nginx (optional)

6.4 Architecture Diagram



7. Database Design

7.1 Schema Overview

Table: users

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Table: account_types

```
CREATE TABLE account_types (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(50) UNIQUE NOT NULL,  
    description TEXT  
);  
  
-- Seed data  
INSERT INTO account_types (name, description) VALUES  
(  
    'bank', 'Bank account',  
    'ewallet', 'E-Wallet (GoPay, OVO, Dana, etc)',  
    'cash', 'Physical cash',  
    'gold', 'Physical gold holdings',  
    'stock_broker', 'Stock brokerage account'  
);
```

Table: accounts

```
CREATE TABLE accounts (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
    account_type_id INT NOT NULL REFERENCES account_types(id),  
    name VARCHAR(100) NOT NULL,  
    balance NUMERIC(15,2) NOT NULL DEFAULT 0,  
    currency VARCHAR(3) NOT NULL DEFAULT 'IDR',  
    is_active BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_accounts_user_id ON accounts(user_id);
```

Table: categories

```
CREATE TABLE categories (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
    name VARCHAR(50) NOT NULL,  
    type VARCHAR(10) NOT NULL CHECK (type IN ('income', 'expense')),  
    is_default BOOLEAN NOT NULL DEFAULT FALSE,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_categories_user_id ON categories(user_id);
```

```
-- Seed default categories
INSERT INTO categories (name, type, is_default) VALUES
('Salary', 'income', TRUE),
('Freelance', 'income', TRUE),
('Investment', 'income', TRUE),
('Food', 'expense', TRUE),
('Transport', 'expense', TRUE),
('Shopping', 'expense', TRUE),
('Bills', 'expense', TRUE),
('Entertainment', 'expense', TRUE);
```

Table: transactions

```
CREATE TABLE transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  account_id UUID NOT NULL REFERENCES accounts(id) ON DELETE CASCADE,
  category_id UUID REFERENCES categories(id) ON DELETE SET NULL,
  type VARCHAR(10) NOT NULL CHECK (type IN ('income', 'expense', 'transfer')),
  amount NUMERIC(15,2) NOT NULL,
  description TEXT,
  date TIMESTAMPTZ NOT NULL,
  transfer_account_id UUID REFERENCES accounts(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_transactions_user_id ON transactions(user_id);
CREATE INDEX idx_transactions_account_id ON transactions(account_id);
CREATE INDEX idx_transactions_date ON transactions(date);
```

8. API Specification

8.1 Base URL

```
http://localhost:8080/api/v1
```

8.2 Authentication

POST /auth/register

Register new user.

Request:

```
{
  "email": "user@example.com",
  "password": "securepassword"
}
```

Response (201):

```
{
  "id": "uuid",
  "email": "user@example.com",
  "created_at": "2026-06-03T10:00:00Z"
}
```

POST /auth/login

Login user.

Request:

```
{
  "email": "user@example.com",
  "password": "securepassword"
}
```

Response (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "uuid",
    "email": "user@example.com"
  }
}
```


8.3 Accounts

All endpoints require `Authorization: Bearer <token>` header.

GET /accounts

Get all user accounts.

Response (200):

```
{
  "accounts": [
    {
      "id": "uuid",
      "name": "BCA Savings",
      "type": "bank",
      "balance": 5000000.00,
      "currency": "IDR",
      "is_active": true
    }
  ]
}
```

POST /accounts

Create new account.

Request:

```
{
  "name": "BCA Savings",
  "account_type_id": 1,
  "balance": 5000000.00
}
```

Response (201):

```
{
  "id": "uuid",
  "name": "BCA Savings",
  "type": "bank",
  "balance": 5000000.00,
  "currency": "IDR",
  "is_active": true,
}
```

```
"created_at": "2026-06-03T10:00:00Z"
}
```

PUT /accounts/:id

Update account.

Request:

```
{
  "name": "BCA Checking",
  "is_active": true
}
```

Response (200):

```
{
  "id": "uuid",
  "name": "BCA Checking",
  "type": "bank",
  "balance": 5000000.00,
  "currency": "IDR",
  "is_active": true,
  "updated_at": "2026-06-03T11:00:00Z"
}
```

DELETE /accounts/:id

Soft delete account (set is_active = false).

Response (204): No content

8.4 Transactions

GET /transactions

Get all transactions with optional filters.

Query Params: - `start_date` (optional): YYYY-MM-DD - `end_date` (optional): YYYY-MM-DD - `account_id` (optional): UUID - `category_id` (optional): UUID - `type` (optional): income | expense | transfer

Response (200):

```
{
  "transactions": [
    {
      "id": "uuid",
      "account": {
        "id": "uuid",
        "name": "BCA Savings"
      },
      "category": {
        "id": "uuid",
        "name": "Food"
      },
      "type": "expense",
      "amount": 50000.00,
      "description": "Lunch",
      "date": "2026-06-03T12:00:00Z"
    }
  ]
}
```

POST /transactions

Create new transaction.

Request (Income/Expense):

```
{
  "account_id": "uuid",
  "category_id": "uuid",
  "type": "expense",
  "amount": 50000.00,
  "description": "Lunch",
  "date": "2026-06-03T12:00:00Z"
}
```

Request (Transfer):

```
{
  "account_id": "uuid",
  "transfer_account_id": "uuid",
  "type": "transfer",
  "amount": 100000.00,
  "description": "Transfer to savings",
  "date": "2026-06-03T12:00:00Z"
}
```

Response (201):

```
{
  "id": "uuid",
  "type": "expense",
  "amount": 50000.00,
  "description": "Lunch",
  "date": "2026-06-03T12:00:00Z",
  "created_at": "2026-06-03T12:01:00Z"
}
```

8.5 Reports

GET /reports/net-worth

Get total net worth (sum of all active accounts).

Response (200):

```
{
  "net_worth": 10500000.00,
  "accounts": [
    {
      "name": "BCA Savings",
      "balance": 5000000.00
    },
    {
      "name": "GoPay",
      "balance": 500000.00
    }
  ]
}
```

GET /reports/spending-by-category

Get spending breakdown by category.

Query Params: - `month` (optional): YYYY-MM (default: current month)

Response (200):

```
{
  "month": "2026-06",
  "total_spending": 2500000.00,
  "categories": [
```

```
{
  "name": "Food",
  "amount": 1000000.00,
  "percentage": 40.0
},
{
  "name": "Transport",
  "amount": 500000.00,
  "percentage": 20.0
}
]
```

9. Frontend Structure

9.1 Pages

- `/login` - Login page
- `/register` - Register page
- `/dashboard` - Main dashboard (net worth + quick stats)
- `/accounts` - Account list and management
- `/transactions` - Transaction list and filters
- `/transactions/new` - Add new transaction
- `/reports` - Reports and analytics

9.2 State Management (Zustand)

Stores: - `authStore` - User authentication state - `accountStore` - Accounts data - `transactionStore` - Transactions data - `reportStore` - Reports data

9.3 Components Structure

```
src/
├── components/
│   ├── Layout/
│   ├── Dashboard/
│   ├── Accounts/
│   ├── Transactions/
│   └── Reports/
├── stores/
└── services/
```

```
|   └─ api.ts
|   └─ types/
|   └─ pages/
```

10. Implementation Roadmap

Phase 0: Project Setup (Week 1)

- [] Initialize backend repository with Go modules
- [] Setup Clean Architecture folder structure
- [] Setup Docker Compose (PostgreSQL + Backend)
- [] Install and configure golang-migrate
- [] Setup environment configuration
- [] Create Makefile for development

Phase 1: Authentication & Foundation (Week 1-2)

- [] Create migration for `users` table
- [] Implement User entity and repository
- [] Implement Register usecase and handler
- [] Implement Login usecase and handler
- [] Implement JWT middleware
- [] Setup error handling standards
- [] Create migration for `account_types` with seed data

Phase 2: Account Management (Week 2-3)

- [] Create migration for `accounts` table
- [] Implement Account entity and repository
- [] Implement Create Account usecase
- [] Implement List Accounts usecase
- [] Implement Update Account usecase
- [] Implement Soft Delete Account usecase
- [] Create API handlers for all account endpoints

Phase 3: Transaction Management (Week 3-4)

- [] Create migration for `categories` with seed data

- [] Create migration for `transactions` table
- [] Implement Transaction entity and repository
- [] Implement Create Income/Expense transaction usecase
- [] Implement Transfer transaction usecase (update 2 balances)
- [] Implement List Transactions with filters usecase
- [] Create API handlers for transaction endpoints

Phase 4: Reporting (Week 4-5)

- [] Implement Net Worth calculation usecase
- [] Implement Spending by Category usecase
- [] Create API handlers for report endpoints
- [] Add date range filtering for reports

Phase 5: Web Frontend (Week 5-7)

- [] Setup React + TypeScript + Tailwind project
- [] Setup Zustand stores
- [] Create auth pages (Login/Register)
- [] Implement Dashboard page (Net Worth + Account balances)
- [] Implement Accounts page (list + create + edit)
- [] Implement Transactions page (list + create + filter)
- [] Implement Reports page (charts + breakdown)
- [] Connect all pages to backend API

Phase 6: Deployment (Week 7)

- [] Create production Docker Compose file
 - [] Setup PostgreSQL backup script
 - [] Deploy to Proxmox VE server
 - [] Configure Nginx reverse proxy (optional)
 - [] Setup SSL certificate (optional)
-

11. Deployment

11.1 Docker Compose Structure

```
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: fintrack
      POSTGRES_USER: fintrack
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  backend:
    build: ./fintrack-backend
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_NAME: fintrack
      DB_USER: fintrack
      DB_PASSWORD: ${DB_PASSWORD}
      JWT_SECRET: ${JWT_SECRET}
    ports:
      - "8080:8080"
    depends_on:
      - postgres

  web:
    build: ./fintrack-web
    ports:
      - "3000:80"
    depends_on:
      - backend

volumes:
  postgres_data:
```

11.2 Environment Variables

Create `.env` file:


```
DB_PASSWORD=your_secure_password
JWT_SECRET=your_jwt_secret_key
```

12. Open Questions & Decisions Log

Decisions Made

- **2026-06-03:** Decided to use Go + Gin for backend (performance + simplicity)
- **2026-06-03:** Chose Clean Architecture for better maintainability
- **2026-06-03:** v1 will focus on FIAT only, crypto deferred to v2
- **2026-06-03:** Web frontend prioritized over mobile app
- **2026-06-03:** IDR only for v1, multi-currency in v2
- **2026-06-03:** Transfer transactions update both balances in single transaction

Open Questions

- Should we add transaction attachments (receipt images) in v1?
- Do we need soft delete for transactions or hard delete is fine?
- Should we add expense categories limit/budget in v1?

13. Future Enhancements (v2+)

Version 2

- Cryptocurrency portfolio tracking
- React Native mobile app
- Multi-currency support
- Recurring transactions
- Budget tracking per category

Version 3

- Bank API integration (open banking)
- Import transactions from CSV/Excel
- Export reports to PDF

- Shared accounts (family mode)
 - Investment portfolio tracking (stocks detail)
-

Document End

This document is a living specification and will be updated as the project evolves.